ALTERNATIVE ASSESSMENT TEST – 1

PORTFOLIO-DRIVEN DEVELOPMENT PROJECT REPORT

Student Name: _____
USN: _____
Semester / Section: _____
Subject: _____
Academic Year: 2026–27
Project Title: **AttendWise — Smart Attendance Planner**
GitHub Repository Link: https://github.com/K-P-Videsh-Kumaar/Attendwise

1. PROJECT TITLE

Project Name: AttendWise — Smart Attendance Planner

2. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

Students often learn that their attendance is below the required threshold too late to make a practical recovery plan. Existing college portals usually present a static percentage, but do not let students safely understand how an attended or missed future class will affect that number. AttendWise solves this as a student-managed planning tool. It records attendance locally through an authenticated web application, identifies subjects that need attention, and provides a read-only what-if simulator. It does not claim to replace an official college attendance record.

Objectives

1. Allow a student to create an account and manage subjects with configurable attendance targets.
2. Record attended, absent and cancelled classes while preventing duplicate subject/date/session entries.
3. Calculate accurate attendance percentages that exclude cancelled classes.
4. Provide risk indicators, recovery requirements and safe-absence estimates.
5. Let a student simulate future attend/skip decisions without changing saved data.
6. Deliver a responsive, polished frontend with clear visual feedback and accessible actions.

3. TECHNOLOGIES / TOOLS USED

Sl. No.	Technology / Tool	Purpose
1	Python + FastAPI	REST API, validation and interactive API

Sl. No.	Technology / Tool	Purpose
		documentation.
2	SQLAlchemy + SQLite	Portable persistence layer and local development database.
3	React + Vite	Responsive single-page frontend and fast development workflow.
4	Framer Motion	Purposeful page, card, modal, progress and feedback animations.
5	JWT + Argon2 password hashing	Authenticated access and secure credential storage.
6	Pytest	Automated tests for attendance and planning rules.

4. PROJECT DESCRIPTION

AttendWise is a smart attendance planner designed for students. After registration, a student adds subjects and enters a starting attendance balance. They can then log individual class outcomes as attended, absent or cancelled. The dashboard shows the current percentage for each subject, total counted classes and risk states: on track, warning, below target or no data.

The signature feature is the what-if planner. A user selects a subject, enters hypothetical future attended and missed classes, and receives a projected percentage. The scenario is calculated server-side and is explicitly read-only. The application also supports a recovery calculation to determine how many consecutive attended classes are needed to reach a configured target.

Expected outcome: students gain an understandable, private planning view of their attendance and can make better decisions before a threshold becomes difficult to recover.

5. IMPLEMENTATION / METHODOLOGY

Step 1: Domain and API design. The project requirements were translated into users, subjects and attendance records, plus protected REST endpoints for authentication, dashboard data and planning.

Step 2: Secure backend foundation. FastAPI, SQLite and SQLAlchemy were configured. Passwords are hashed with Argon2; JWT access tokens protect all user-specific routes; queries enforce ownership at the backend.

Step 3: Attendance calculations. A single service module implements counted-class totals, percentages, simulator projections, recovery requirements and risk descriptions. Cancelled records do not change the denominator.

Step 4: Responsive frontend. The React/Vite interface was created with an editorial dashboard layout, mobile navigation, clear empty states and modal forms. Framer Motion adds subtle transitions to navigation, cards, live progress bars, modals and status toasts.

Step 5: Verification. Unit tests cover cancelled records, simulator non-mutation, recovery edge cases and risk boundaries. The API and frontend build were also verified locally, including CORS for the Vite development origin.

6. IMPORTANT CODE SNIPPETS

Code Snippet 1: Centralised attendance calculation

```
def metrics(baseline_attended, baseline_absent, statuses):
    attended = baseline_attended + sum(
        status == AttendanceStatus.attended for status in statuses
    )
    total = baseline_attended + baseline_absent + sum(
        status in (AttendanceStatus.attended, AttendanceStatus.absent)
        for status in statuses
    )
    return AttendanceMetrics(
        attended, total,
        round(attended / total * 100, 2) if total else None
    )
```

Explanation: This backend service keeps the percentage formula in one tested location. Attended and absent records count toward the denominator; cancelled records do not. The None result for a zero total allows the interface to show “No data” instead of dividing by zero.

Code Snippet 2: Read-only planner endpoint

```
@router.post("/simulate")
def simulate_attendance(payload, user=Depends(current_user), db=Depends(get_db)):
    result = []
    for scenario in payload.scenarios:
        subject = owned_subject(scenario.subject_id, user, db)
        before = subject_metrics(subject, db)
        after = simulate(before, scenario.attended, scenario.missed)
        result.append({
            "subject_id": subject.id,
            "current_percentage": before.percentage,
            "projected_percentage": after.percentage,
        })
    return {"read_only": True, "projections": result}
```

Explanation: The planner authenticates the user, verifies that every requested subject belongs to them, calculates a projection and returns JSON only. It does not insert or update any attendance record.

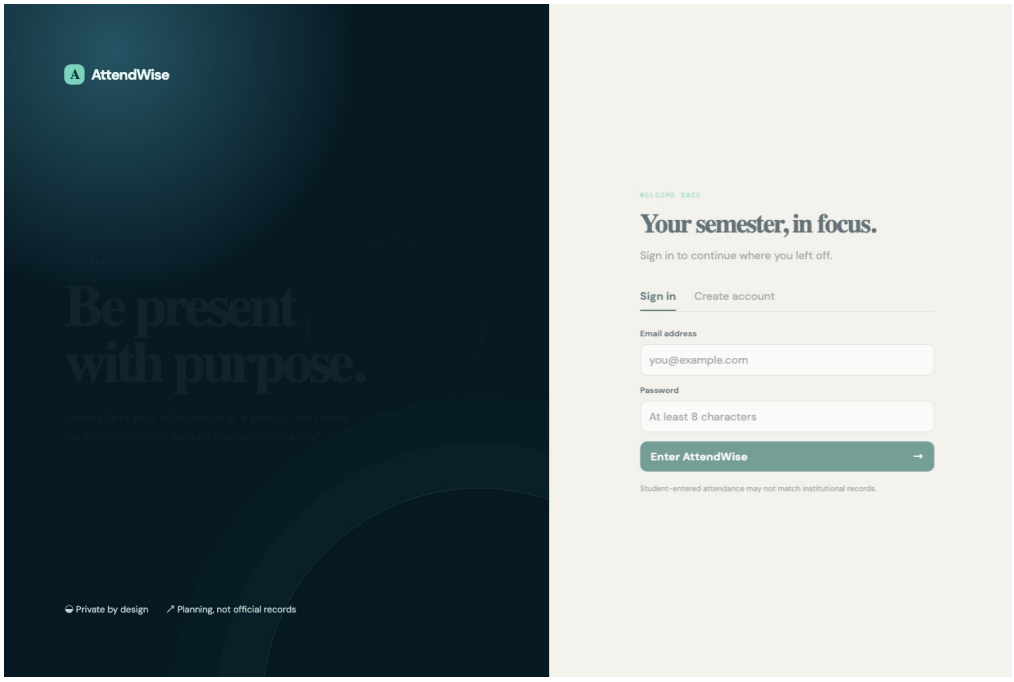
Code Snippet 3: Framer Motion UI feedback

```
<motion.article
  initial={{ opacity: 0, y: 14 }}
  animate={{ opacity: 1, y: 0 }}
  exit={{ opacity: 0, scale: 0.97 }}
  transition={{ delay: index * 0.035 }}
  className="subject-card"
>
  <motion.i
    initial={{ width: 0 }}
    animate={{ width: `${Math.min(percentage || 0, 100)}%` }}
  />
</motion.article>
```

Explanation: Motion is used to make data updates understandable: cards enter gently and each progress indicator grows to its actual attendance percentage. The animation reinforces real information rather than being decorative.

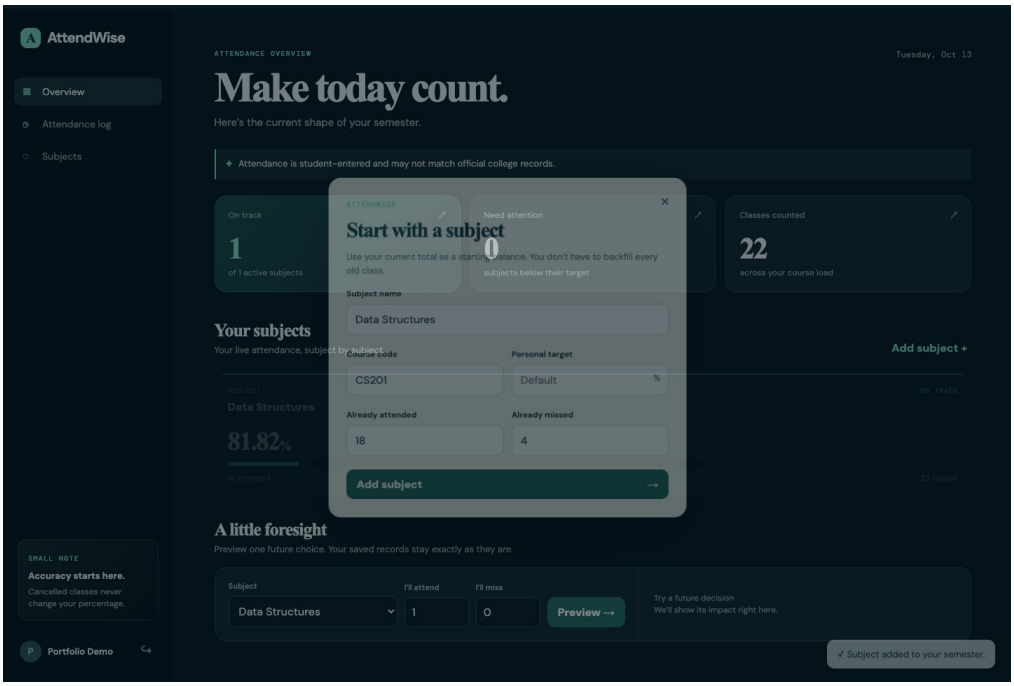
7. OUTPUT SCREENSHOTS

Figure 1: Main Output — AttendWise sign-in screen
The welcome screen presents a focused product proposition and a clear authentication flow.



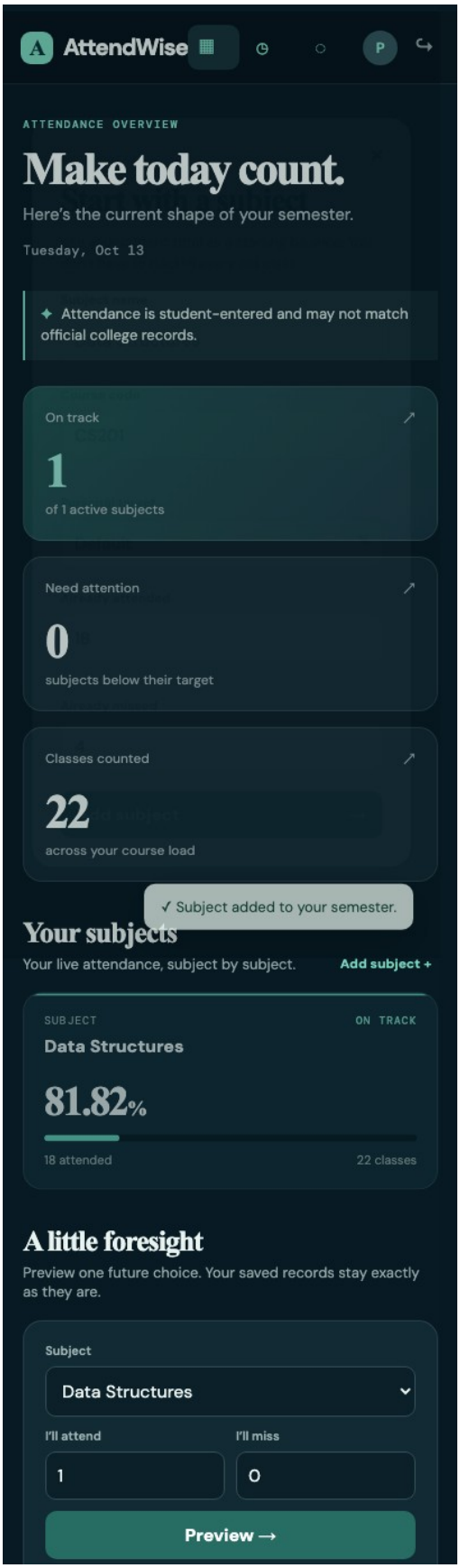
AttendWise sign-in screen

Figure 2: Additional Output / Feature — Dashboard and what-if planner
After creating an account and adding a subject, the dashboard displays subject cards, risk states, animated progress values and the read-only forecast panel.



AttendWise desktop dashboard

Figure 3: Responsive mobile output
The mobile design preserves the hierarchy and core planning actions in a compact layout.


AttendWise
P
ATTENDANCE OVERVIEW
Make today count.
Here's the current shape of your semester.
Tuesday, Oct 13
Attendance is student-entered and may not match official college records.
On track
1
of 1 active subjects
Need attention
0
subjects below their target
Classes counted
22
across your course load
✓ Subject added to your semester.
Your subjects
Your live attendance, subject by subject.
Add subject +
SUBJECT
ON TRACK
Data Structures
81.82%
18 attended
22 classes
A little foresight
Preview one future choice. Your saved records stay exactly as they are.
Subject
Data Structures
I'll attend
I'll miss
1
0
Preview →

AttendWise mobile dashboard

8. GITHUB REPOSITORY SCREENSHOT AND LINK

GitHub Repository Link: https://github.com/K-P-Videsh-Kumaar/Attendwise

Repository Screenshot:
The repository is published at the above link. Its root README contains product screenshots, architecture, setup instructions, validation steps and scope notes for reviewers.

The repository is organised into backend/, frontend/ and report/. The backend holds the FastAPI application, models, schemas, service layer and tests. The frontend contains the Vite configuration, React source and Framer Motion interface. Each main directory contains a README with setup instructions.

9. CHALLENGES FACED

1. **Avoiding inaccurate attendance calculations:** Cancelled classes and zero counted classes required explicit handling. The calculation service excludes cancellations and returns no percentage for empty data.
2. **Protecting student data:** It was necessary to enforce subject and record ownership in every server-side query instead of trusting client-supplied IDs.
3. **Keeping simulation safe:** The planner had to feel interactive without mutating the saved register. The API returns a calculated projection and makes no database write.
4. **Making the interface informative without being overwhelming:** The final UI uses hierarchy, concise language, risk color, responsive layout and restrained Framer Motion transitions to make the state easy to scan.

10. LEARNING OUTCOMES AND CONCLUSION

Learning Outcomes

After completing this project, I learned:

1. How to design a FastAPI application with authentication, SQLAlchemy persistence and Pydantic validation.
2. How to isolate attendance rules in a testable service layer and cover edge cases with automated tests.
3. How to connect a React interface to a protected REST API.
4. How Framer Motion can support product feedback through progressive disclosure, not just visual decoration.
5. How to document an application's scope accurately, including the distinction between student-entered planning data and official attendance records.

Conclusion

AttendWise delivers the core attendance-planning workflow requested for the MVP. A student can add subjects, record attendance outcomes, understand current risk, preview future choices and calculate a recovery path. The project demonstrates a full-stack approach with secure backend ownership rules, tested calculation logic and a responsive, animated frontend. Future work can add timetable slots, forecast ranges, data export and notification preferences.

STUDENT DECLARATION

I hereby declare that the project submitted as part of AAT-1 is my work and that I understand the implementation and contents of the project.

Student Name: _____
Signature: _____
Date: _____